

Welcome

CS 472/572
Unsupervised Machine Learning

Autoencoder

Week-5 -Class -2

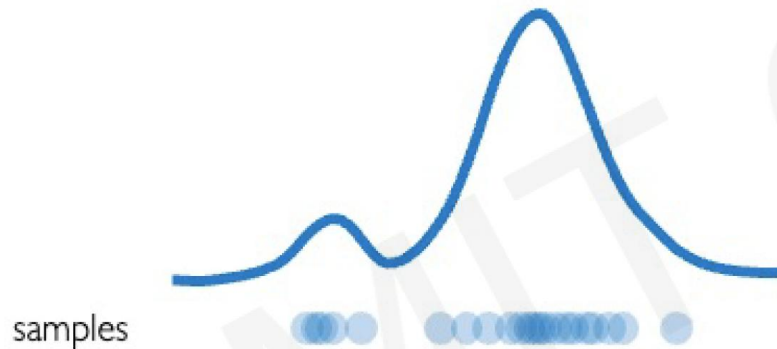
Fall 2025

Instructor: Dr. Khondker Fariha Hossain

Generative modeling

Goal: Take as input training samples from some distribution and learn a model that represents that distribution

Density Estimation



Sample Generation



Input samples

Training data $\sim P_{data}(x)$



Generated samples

Generated $\sim P_{model}(x)$

How can we learn $P_{model}(x)$ similar to $P_{data}(x)$?

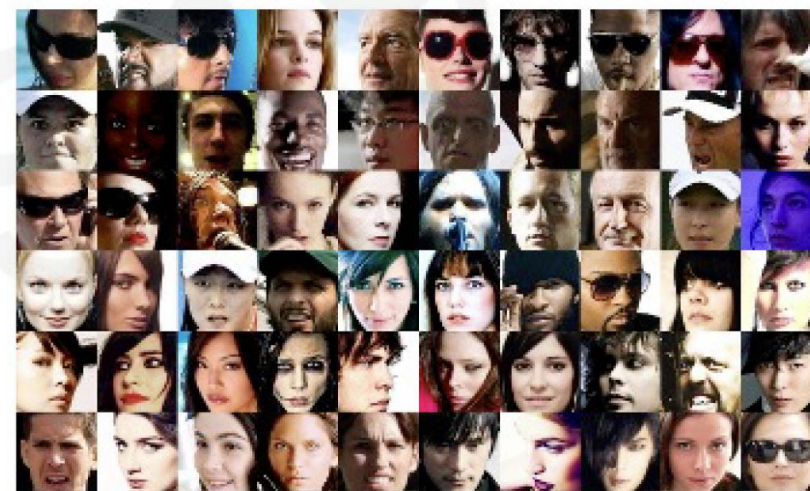
Why Generative models? Debiasing

Capable of uncovering **underlying features** in a dataset



Homogeneous skin color, pose

VS



Diverse skin color, pose, illumination

How can we use this information to create fair and representative datasets?



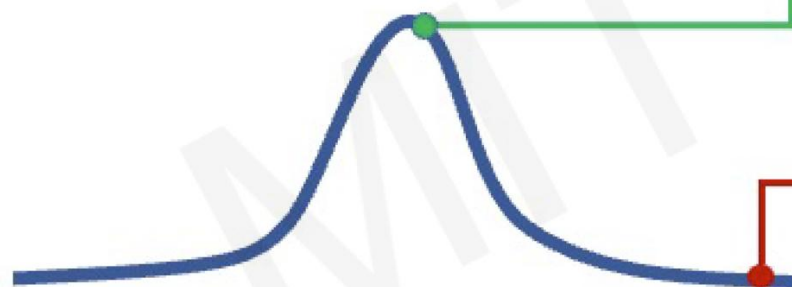
Why Generative models? Outlier detection

- **Problem:** How can we detect when we encounter something new or rare?
- **Strategy:** Leverage generative models, detect outliers in the distribution
- Use outliers during training to improve even more!

95% of Driving Data:
(1) sunny, (2) highway, (3) straight road



Detect outliers to avoid unpredictable behavior when training



Edge Cases



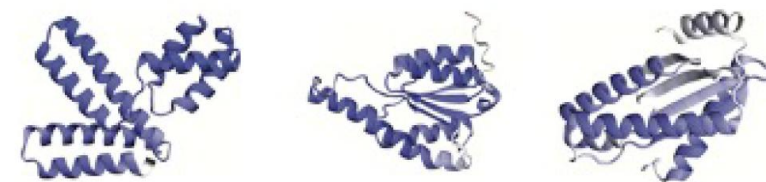
Harsh Weather



Pedestrians

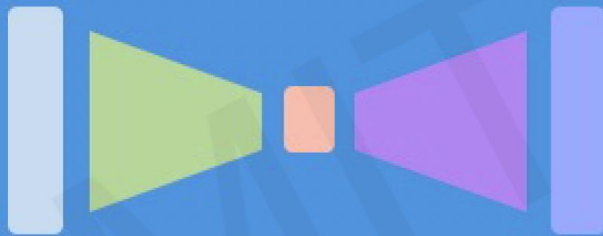
Why Generative models? Sample generation

- **Generative models learn probability distributions**
- **Sampling** from that distribution → new data instances
- **Backbone of Generative AI:** generate language, images, and more

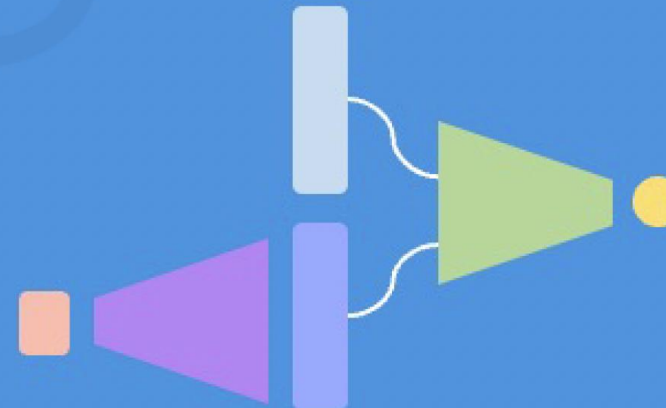


Latent Variable models

Autoencoders and Variational Autoencoders (VAEs)



Generative Adversarial Networks (GANs)



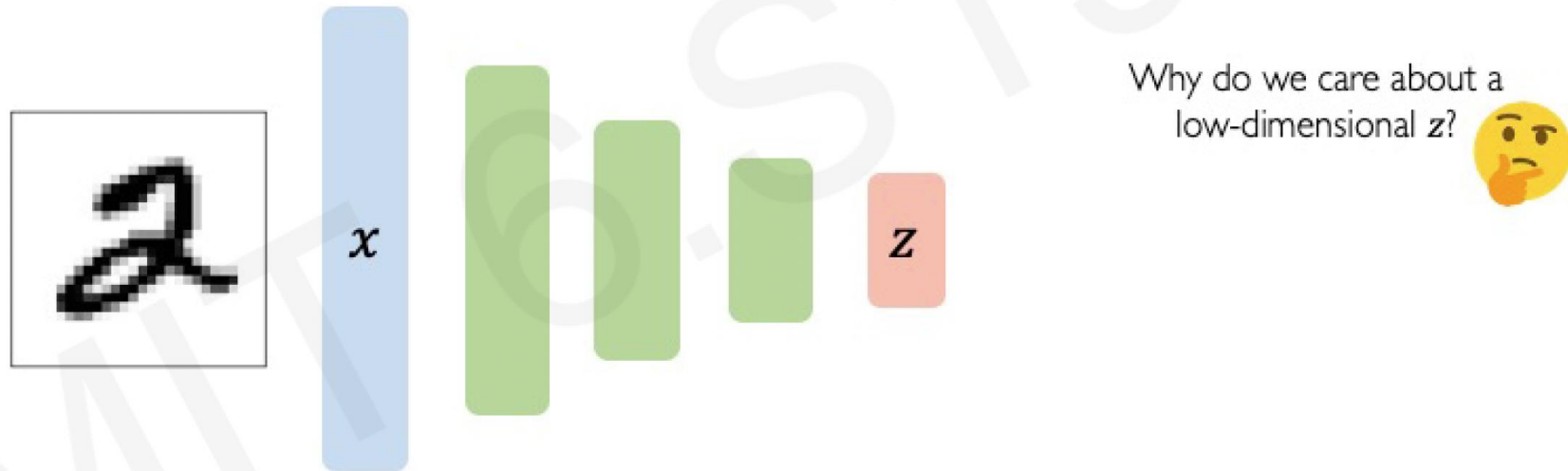
What is Latent variable



Myth of the Cave

Autoencoder: background

Unsupervised approach for learning a **lower-dimensional** feature representation from unlabeled training data

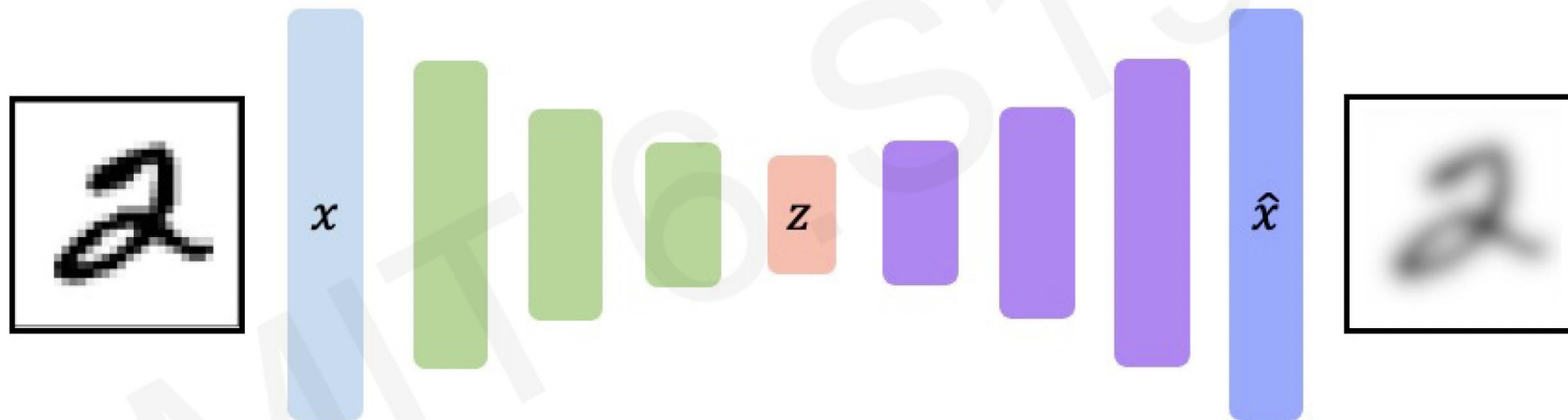


“Encoder” learns mapping from the data, x , to a low-dimensional latent space, z

Autoencoder: background

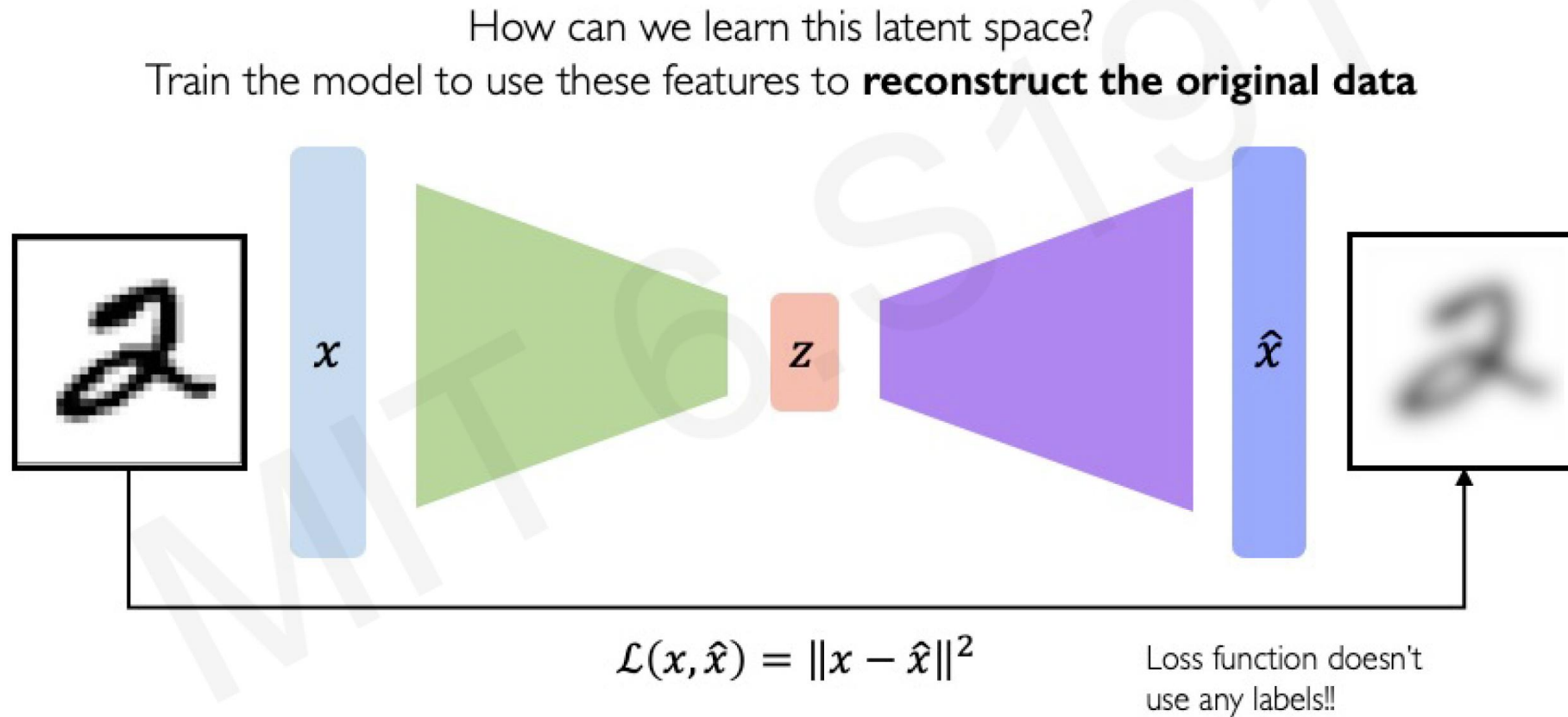
How can we learn this latent space?

Train the model to use these features to **reconstruct the original data**



"Decoder" learns mapping back from latent space, z ,
to a reconstructed observation, $\hat{x}$

Autoencoder: background



Dimensionality of latent space -> reconstruction quality

Autoencoding is a form of compression!
Smaller latent space will force a larger training bottleneck

2D latent space



5D latent space



Ground Truth



Autoencoder for representation learning

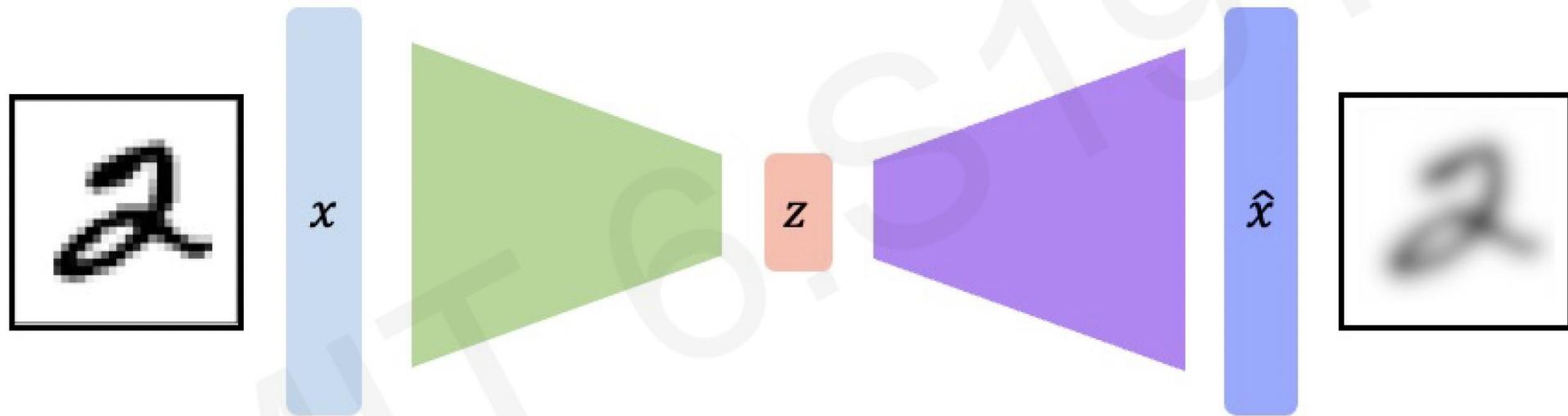
Bottleneck hidden layer forces network to learn a compressed latent representation

Reconstruction loss forces the latent representation to capture (or encode) as much “information” about the data as possible

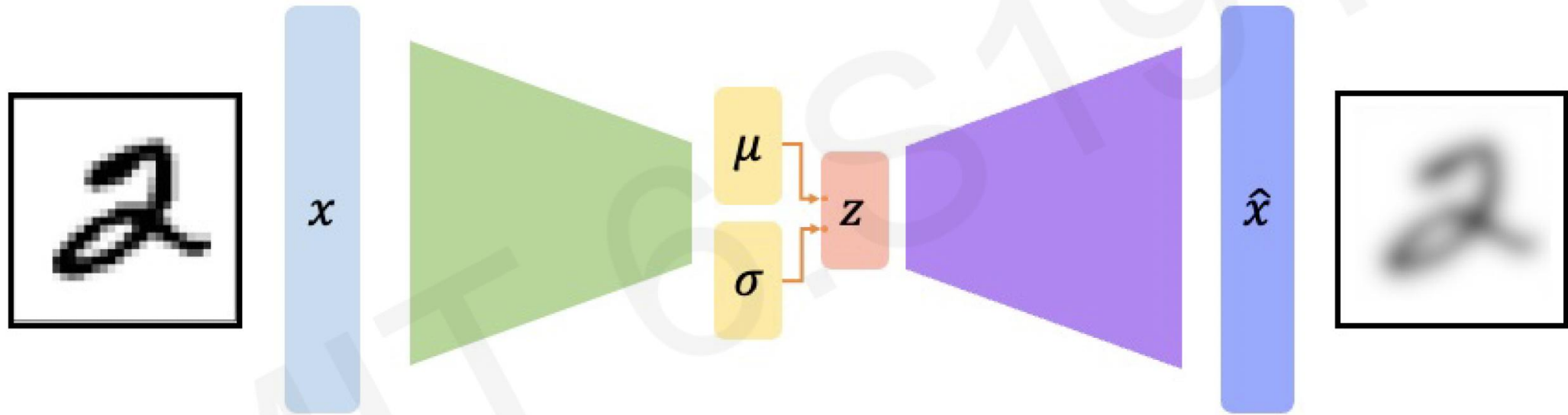
Autoencoding = **Auto** automatically **encoding** data; “Auto” = **self**-encoding

Variational Autoencoder (VAEs)

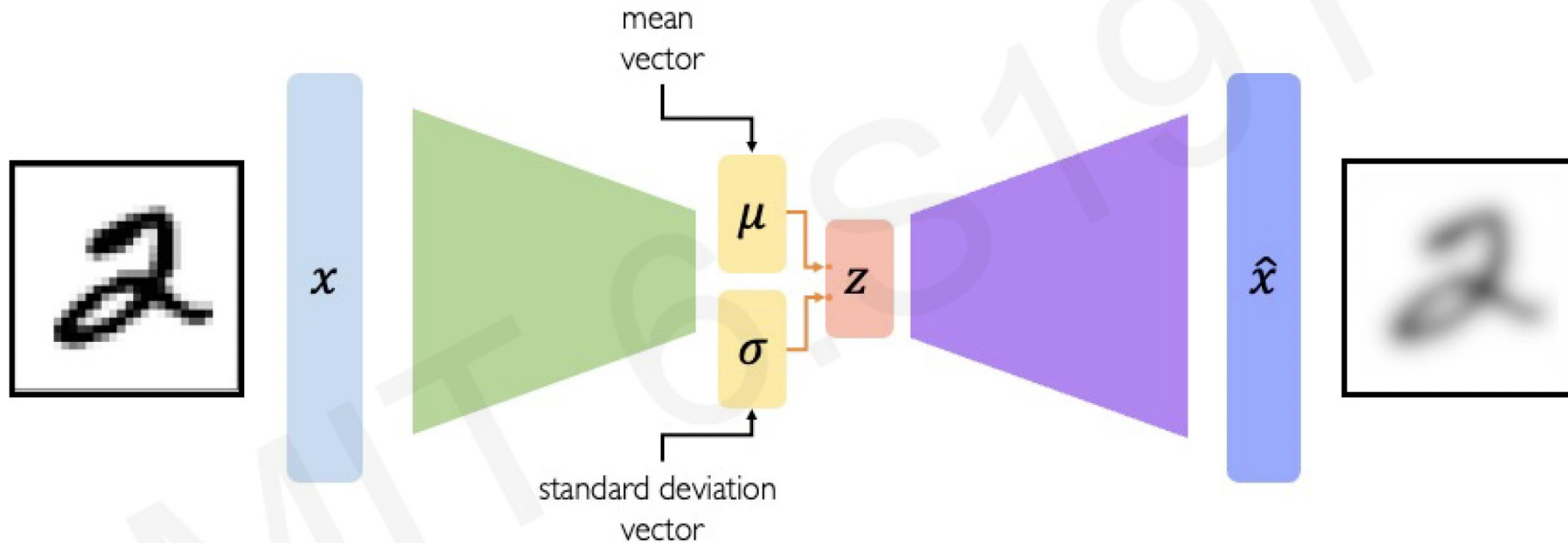
Traditional Autoencoder



VAEs: key difference with traditional autoencoder



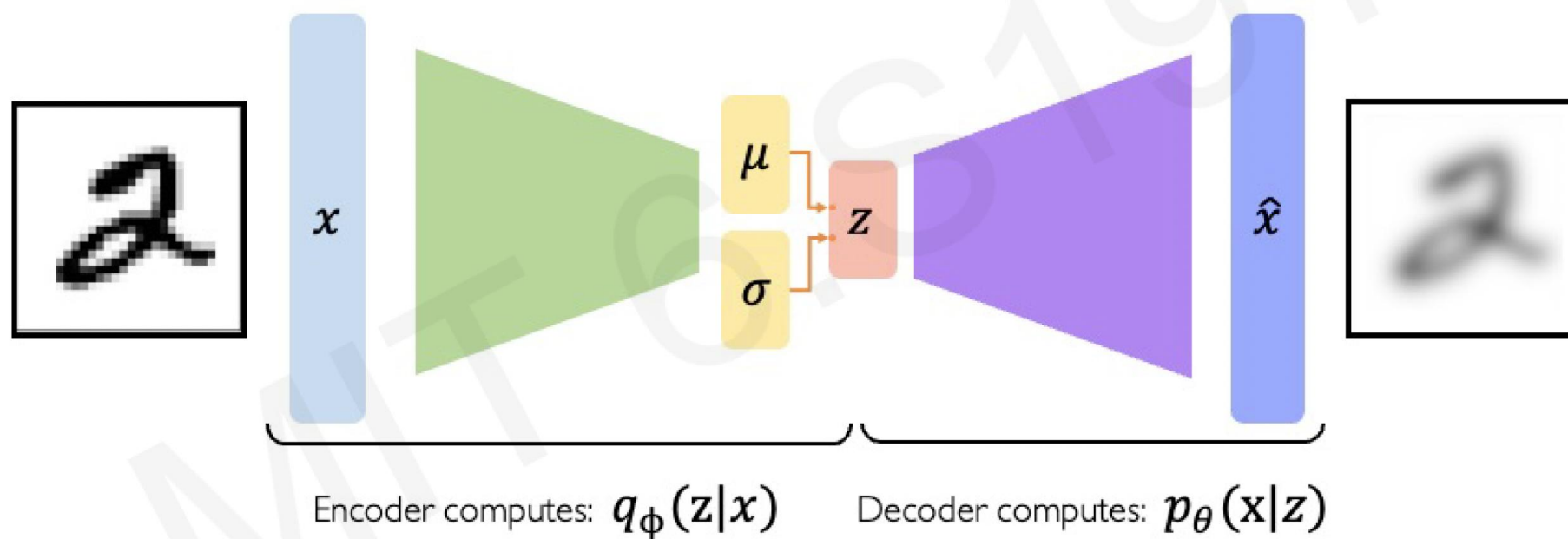
VAEs: key difference with traditional autoencoder



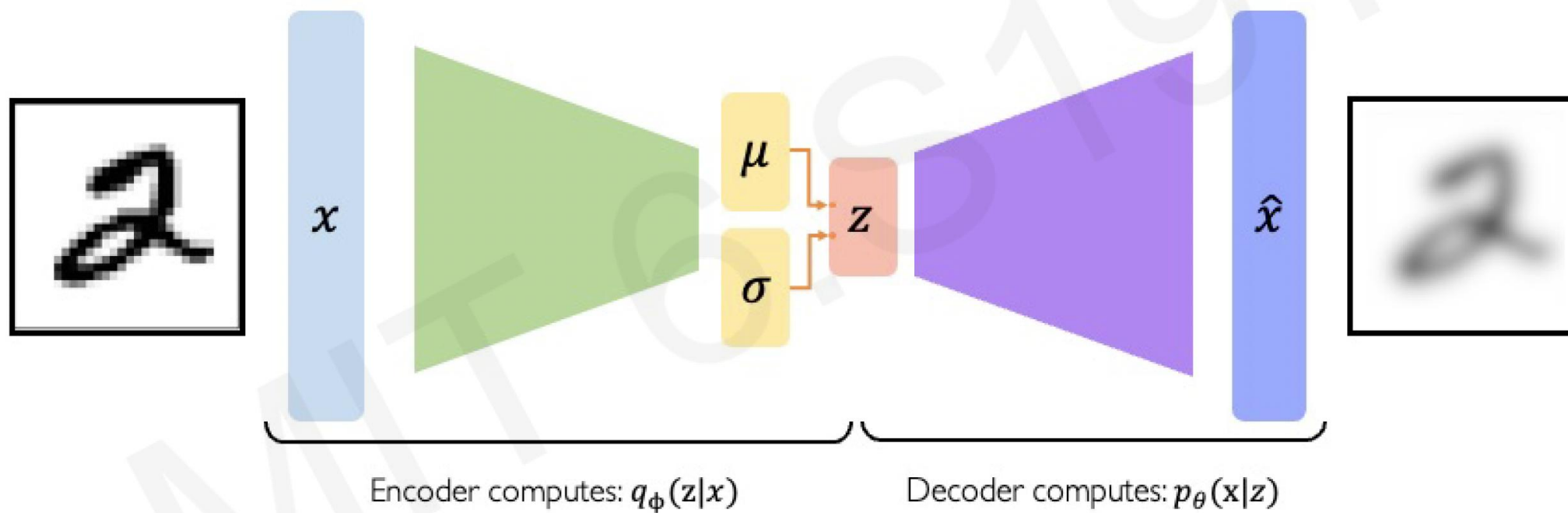
Variational autoencoders are a probabilistic twist on autoencoders!

Sample from the mean and standard deviation to compute latent sample

VAE optimization

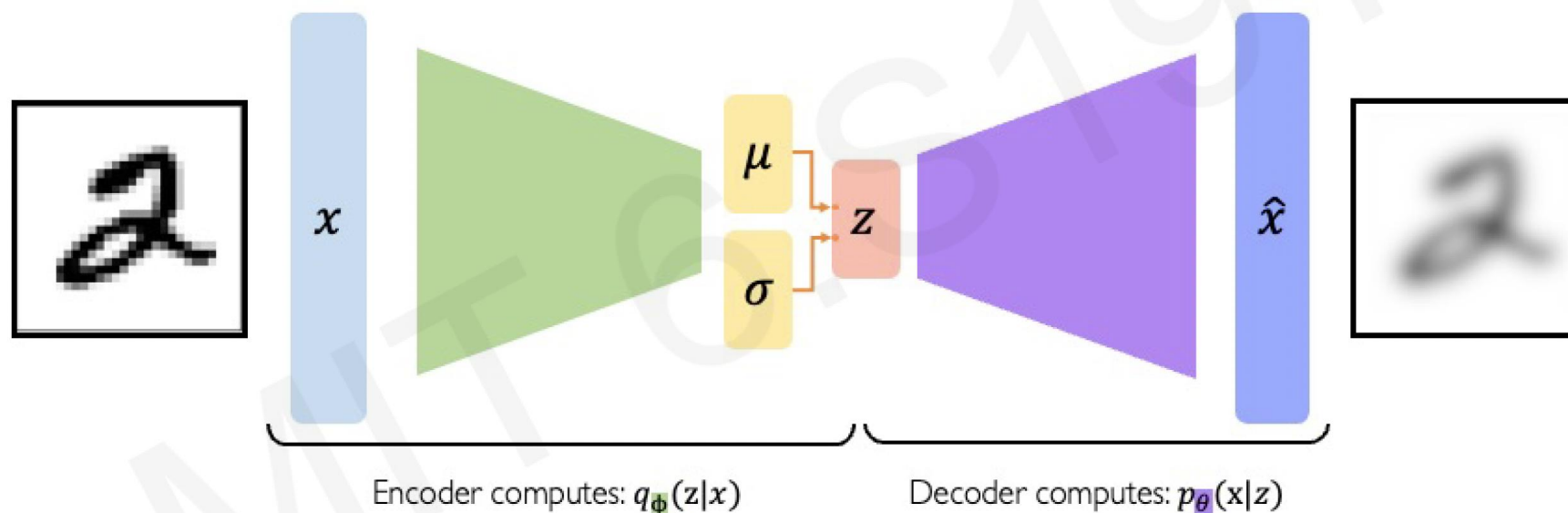


VAE optimization



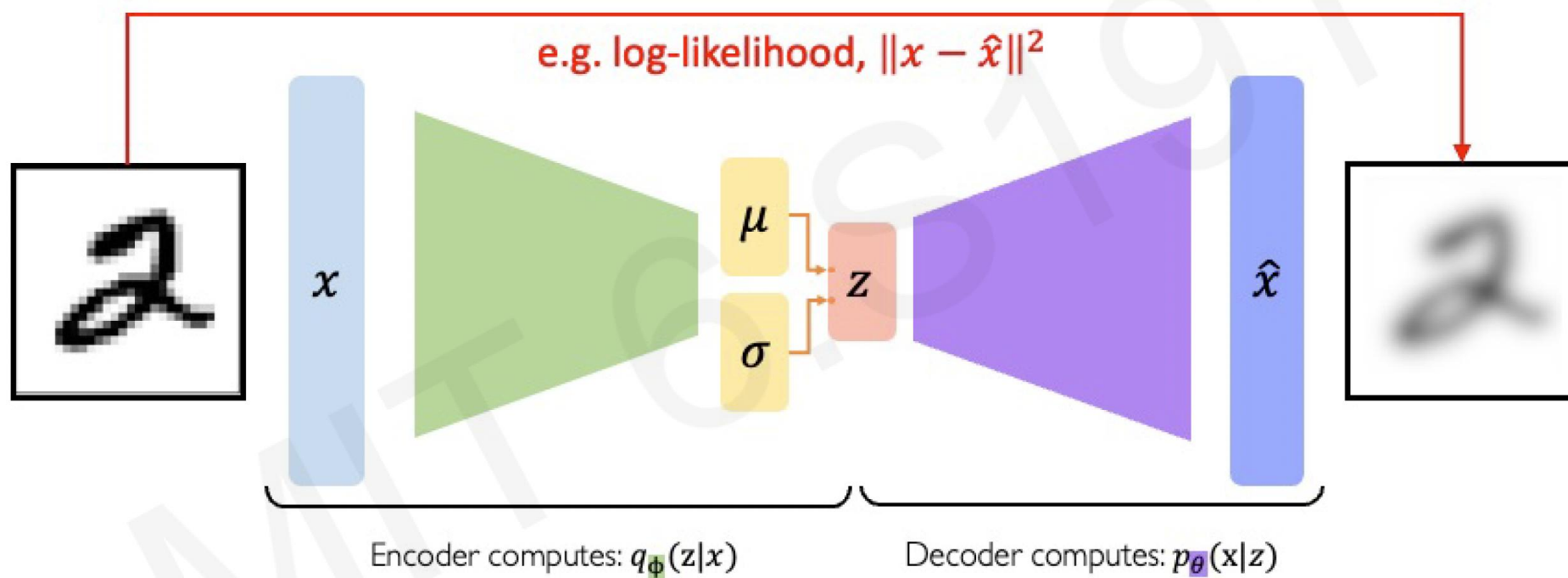
$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

VAE optimization



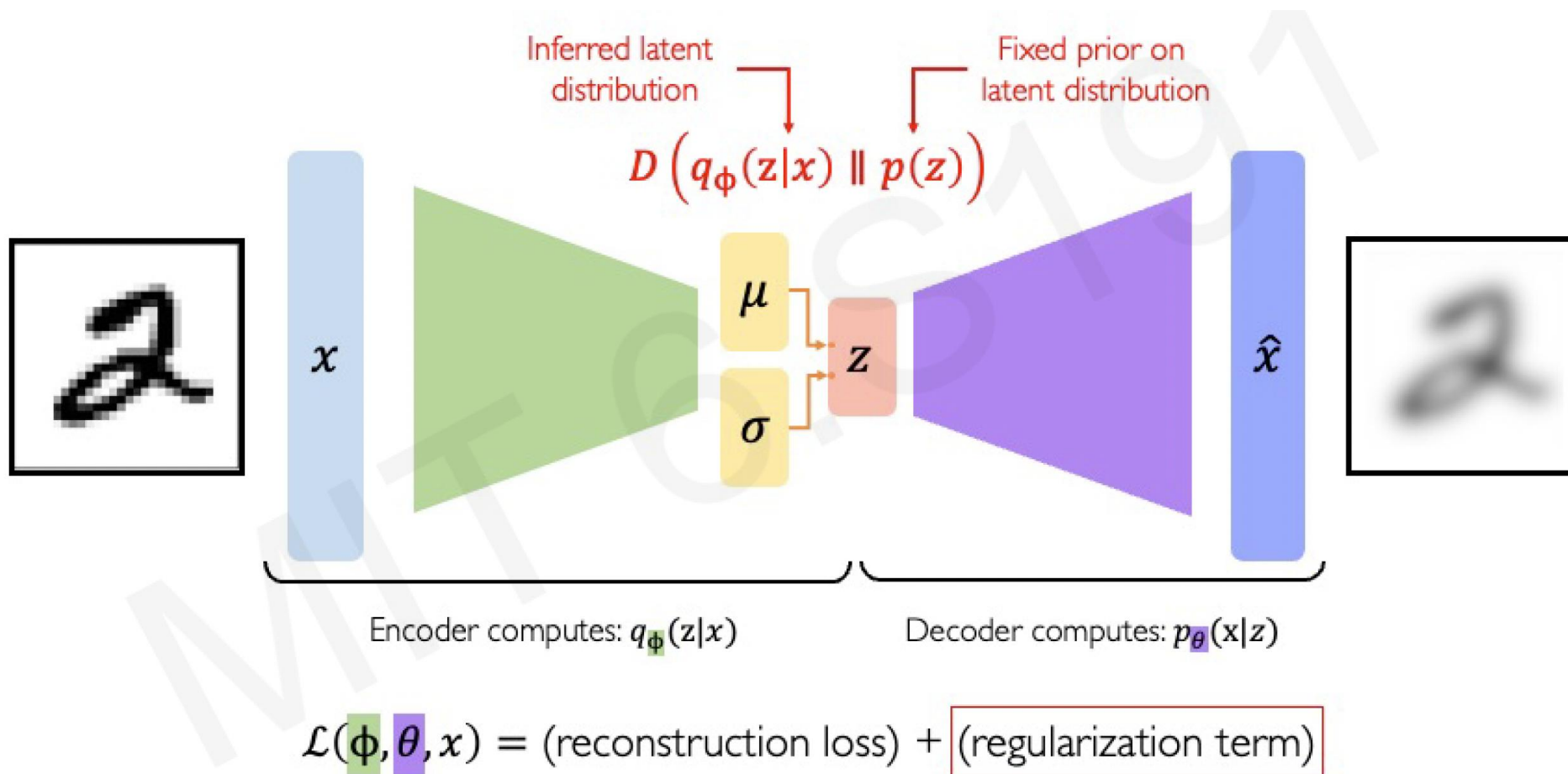
$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

VAE optimization



$$\mathcal{L}(\phi, \theta, x) = \boxed{\text{(reconstruction loss)}} + \text{(regularization term)}$$

VAE optimization

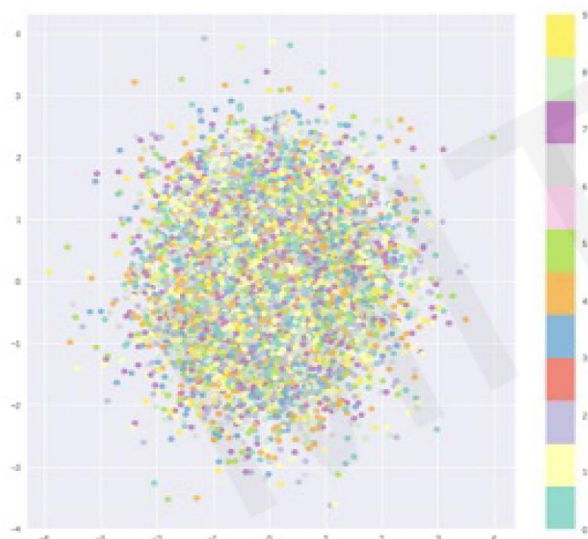


Priors on the latent distribution

$$D \left(q_{\phi}(z|x) \parallel p(z) \right)$$

Inferred latent
distribution

Fixed prior on
latent distribution



Common choice of prior – Normal Gaussian:

$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

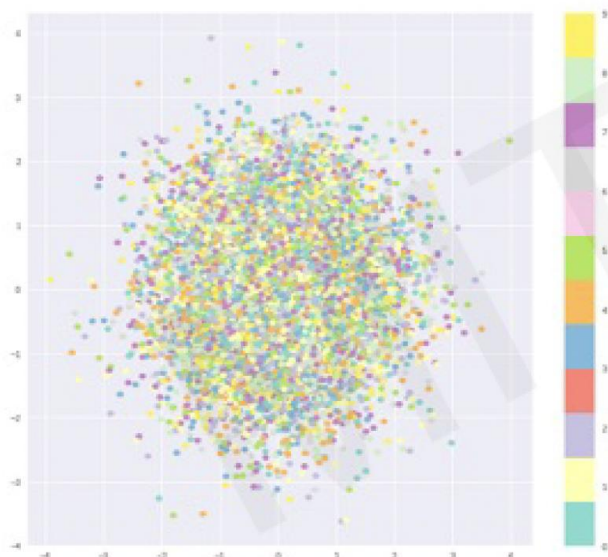
- Encourages encodings to distribute encodings evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (i.e., by memorizing the data)

Priors on the latent distribution

$$D \left(q_{\phi}(z|x) \parallel p(z) \right)$$

$$= -\frac{1}{2} \sum_{j=0}^{k-1} (\sigma_j + \mu_j^2 - 1 - \log \sigma_j)$$

KL-divergence
between the two
distributions



Common choice of prior – Normal Gaussian:

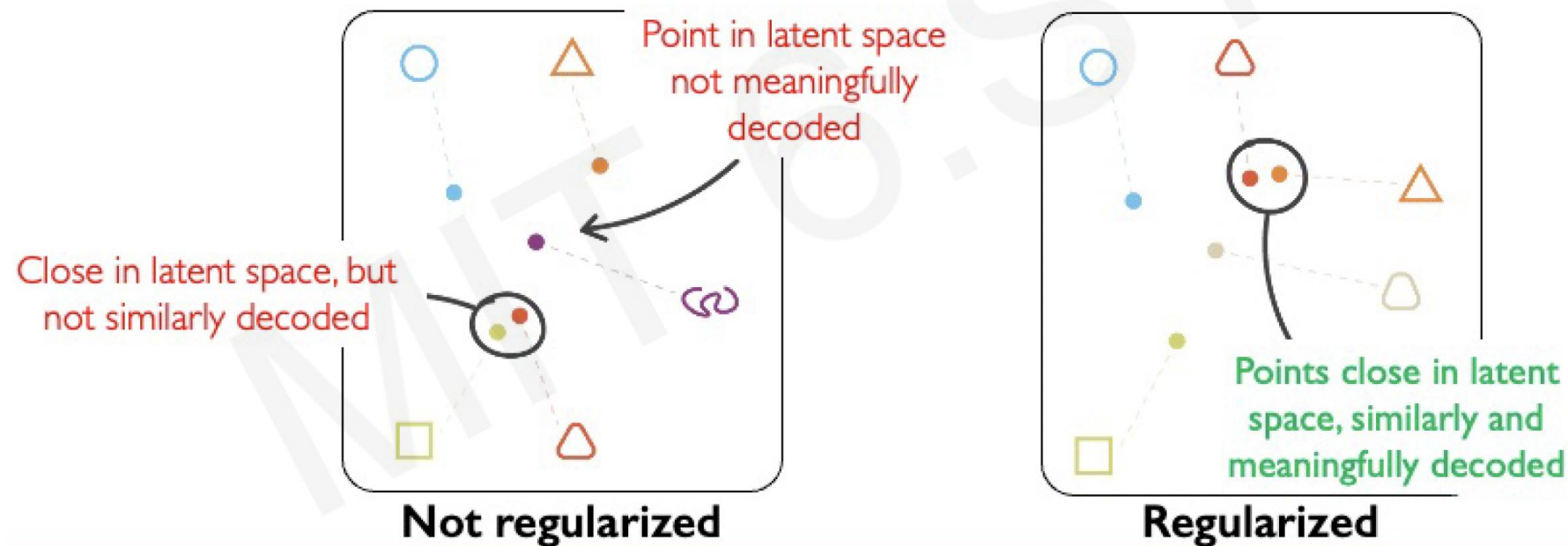
$$p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$$

- Encourages encodings to distribute evenly around the center of the latent space
- Penalize the network when it tries to “cheat” by clustering points in specific regions (i.e., by memorizing the data)

Intuition on regularization and Normal prior

What properties do we want to achieve from regularization? 🤔

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ "meaningful" content after decoding



Intuition on regularization and Normal prior

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ "meaningful" content after decoding

Encoding as a distribution does not guarantee these properties!

Small variances $\rightarrow$
pointed distributions

Different means $\rightarrow$
Discontinuities

Not regularized

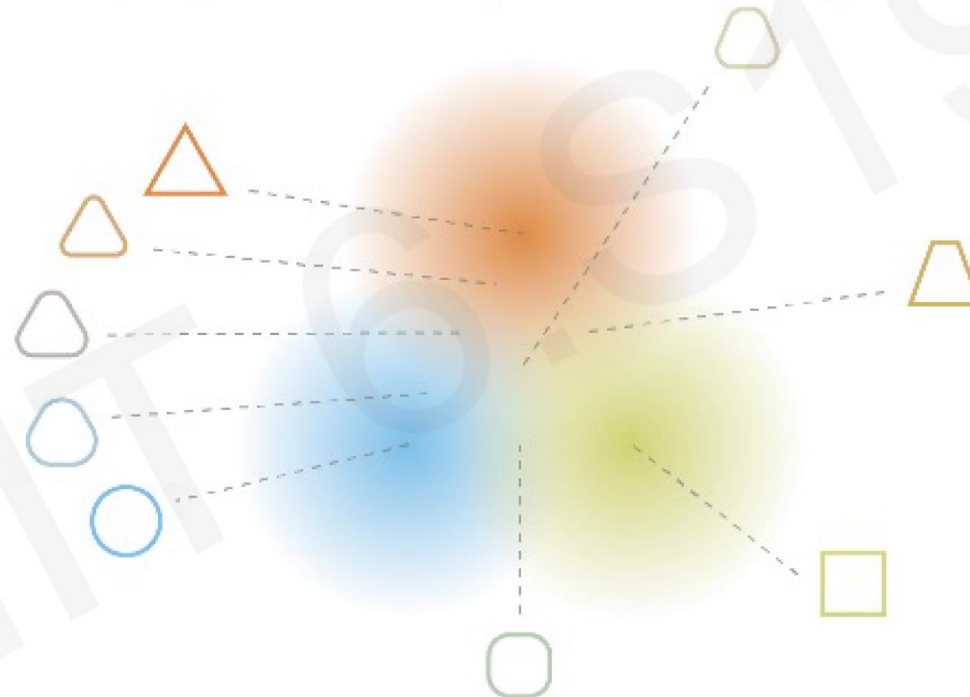
Normal prior $\rightarrow$
continuity + completeness

Center means
Regularize variances

Regularized

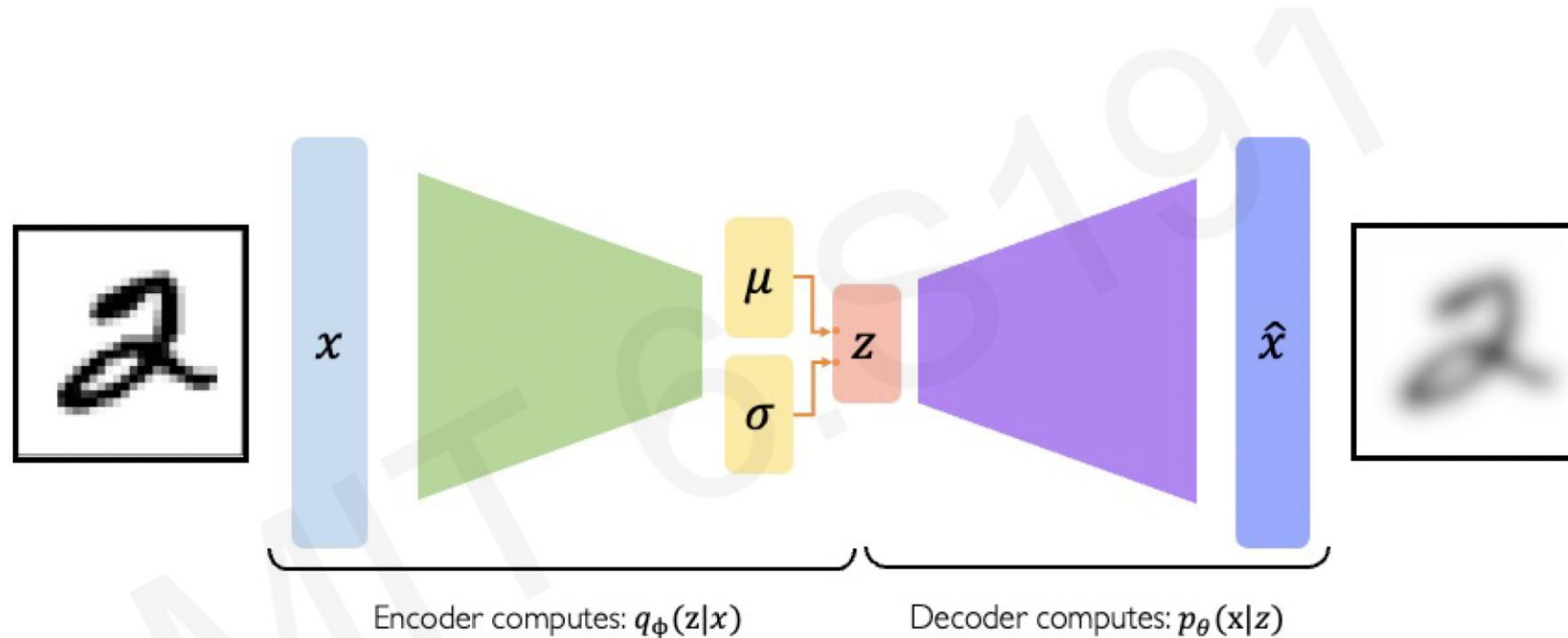
Intuition on regularization and Normal prior

1. **Continuity:** points that are close in latent space $\rightarrow$ similar content after decoding
2. **Completeness:** sampling from latent space $\rightarrow$ "meaningful" content after decoding



Regularization with Normal prior helps enforce **continuity & completeness** in the latent space.

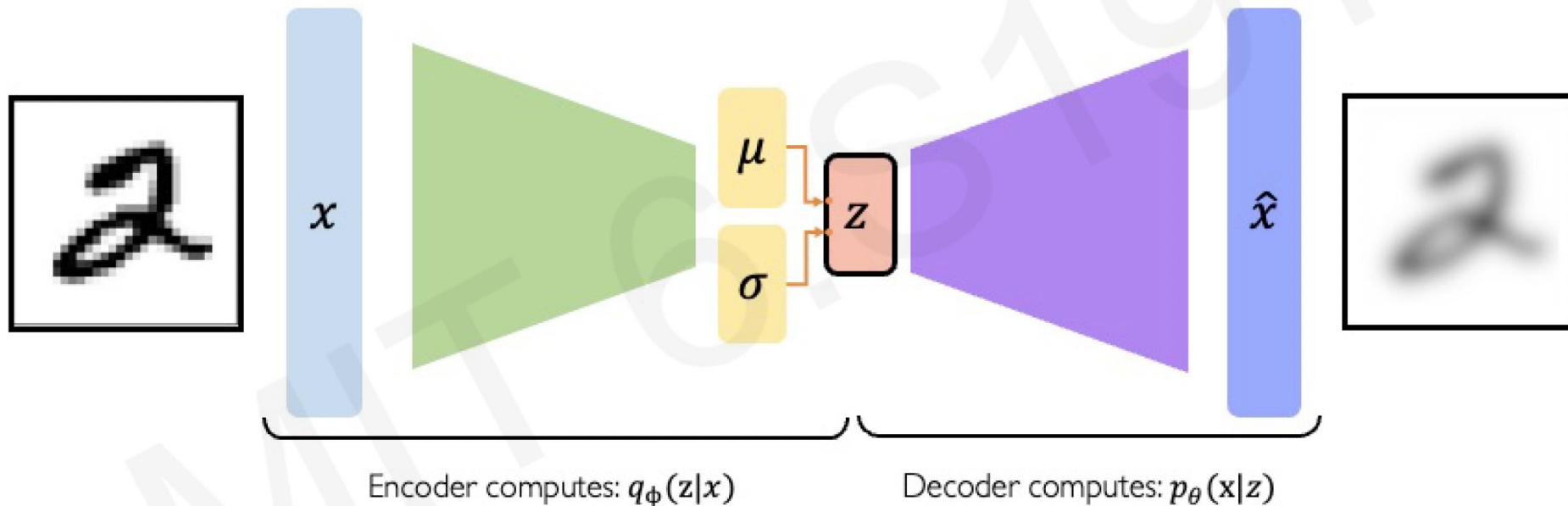
VAE computation graph



$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

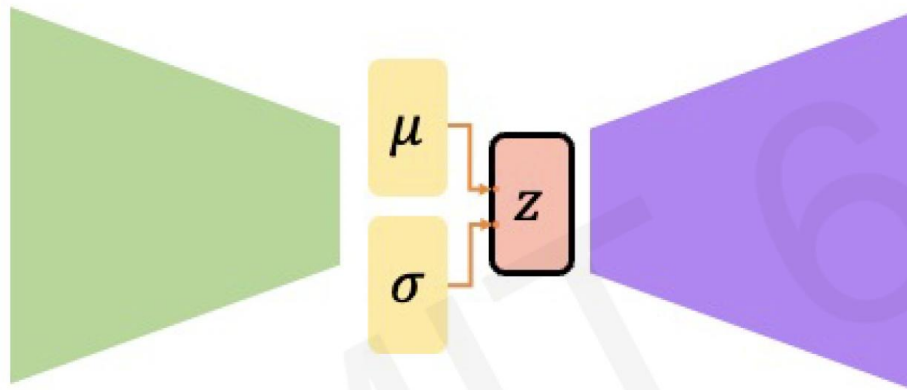
VAE computation graph

Problem: We cannot backpropagate gradients through sampling layers!



$$\mathcal{L}(\phi, \theta, x) = (\text{reconstruction loss}) + (\text{regularization term})$$

Reparametrizing the sampling layer



Key Idea:

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

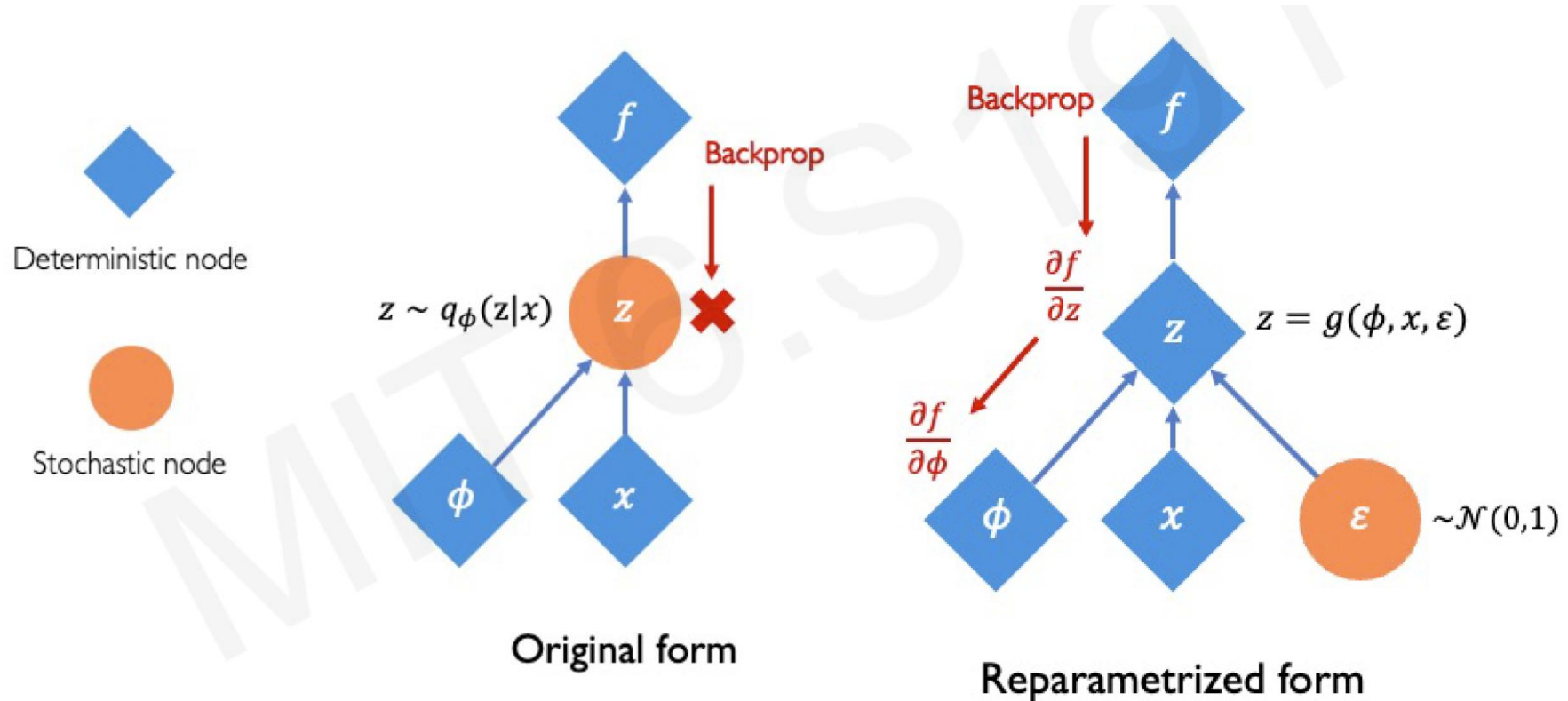
Consider the sampled latent vector z as a sum of

- a fixed μ vector,
- and fixed σ vector, scaled by random constants drawn from the prior distribution

$$\Rightarrow z = \mu + \sigma \odot \epsilon$$

where $\epsilon \sim \mathcal{N}(0,1)$

Reparametrizing the sampling layer



VAEs: Latent perturbation

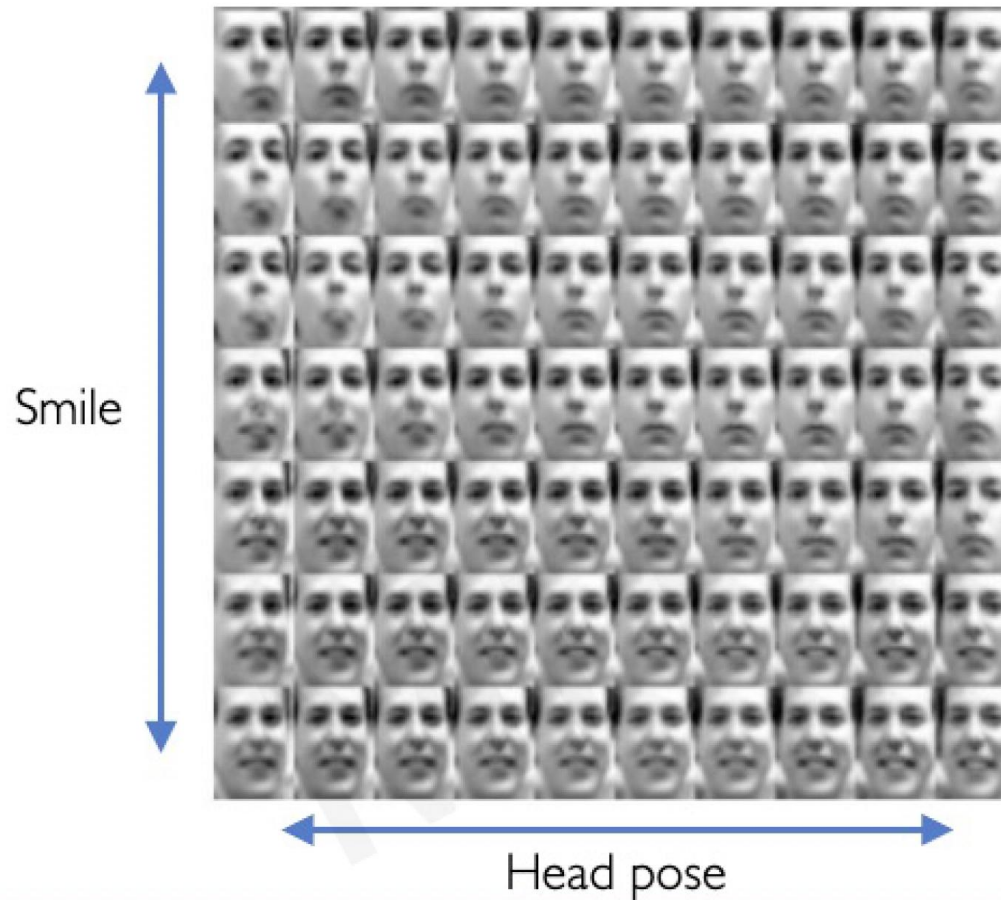
Slowly increase or decrease a **single latent variable**
Keep all other variables fixed



Head pose

Different dimensions of z encodes **different interpretable latent features**

VAEs: Latent perturbation



Ideally, we want latent variables that are uncorrelated with each other

Enforce diagonal prior on the latent variables to encourage independence

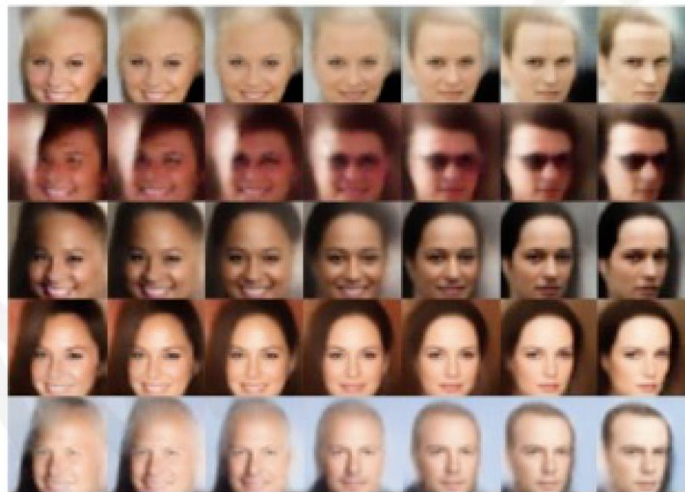
Disentanglement

Latent space disentanglement with β -VAEs

$$\mathcal{L}(\theta, \phi; \mathbf{x}, \mathbf{z}, \beta) = \underbrace{\mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction term}} - \underbrace{\beta D_{KL}(q_{\phi}(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))}_{\text{Regularization term}}$$

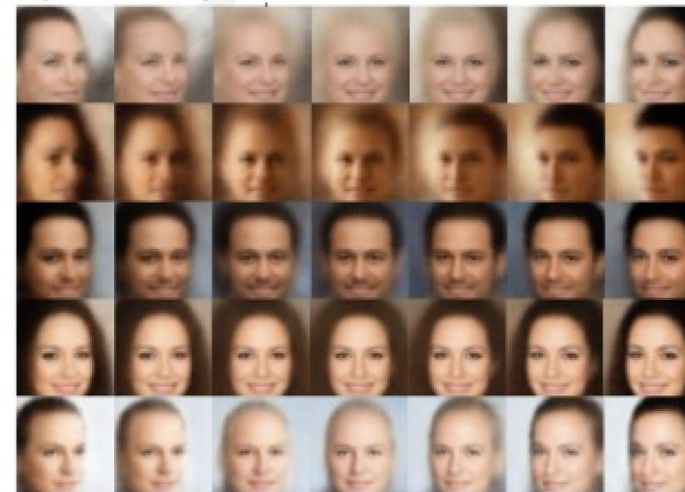
$\beta > 1$: constrain latent bottleneck, encourage efficient latent encoding $\rightarrow$ disentanglement

Head rotation (azimuth)



Standard VAE ($\beta = 1$)

Smile also
changing!



β -VAE ($\beta = 250$)

Smile relatively
constant!

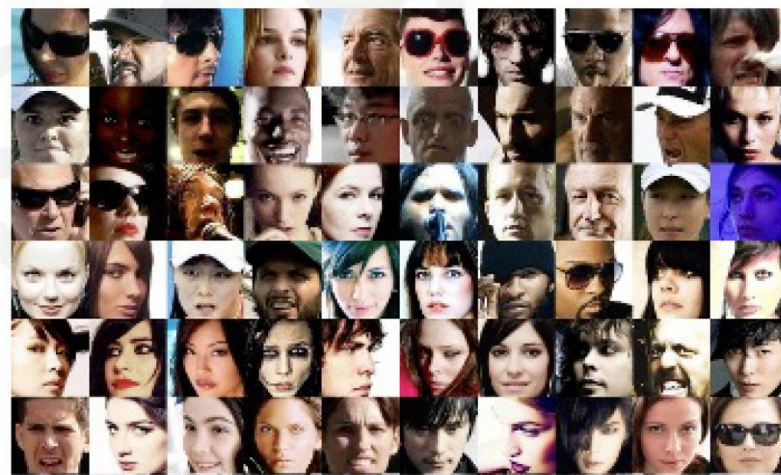
Why latent variable models? Debiasing

Capable of uncovering **underlying latent variables** in a dataset



Homogeneous skin color, pose

VS

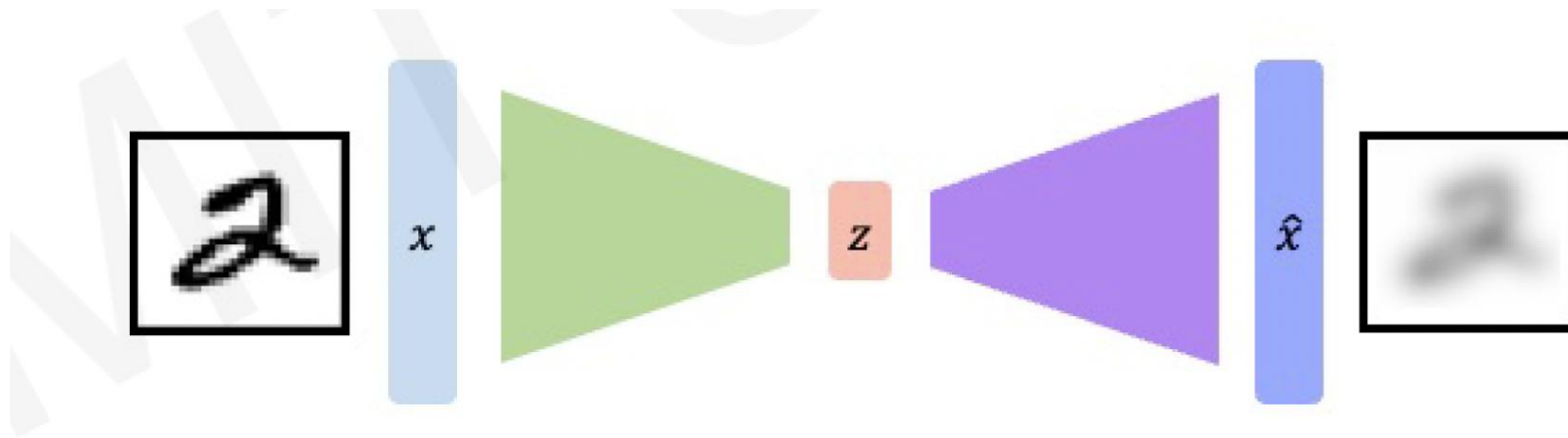


Diverse skin color, pose, illumination

How can we use latent distributions to create fair and representative datasets?

VAE summary

1. Compress representation of world to something we can use to learn
2. Reconstruction allows for unsupervised learning
3. Reparameterization trick to train end-to-end
4. Interpret hidden latent variables using perturbation
5. Generating new examples



Thank You

Reference:

MIT: Introduction to Deep Learning : Deep Generative models